

4 hours examples

Problem 1 (16IOI1)

Sketch:

- Sort the list.
- Find the minimum/maximum sums mi_k, ma_k using k elements.
- If $[mi_k, ma_k]$ and $[L, U]$ overlap, start with the minimum mi_k achieved using the terms 0 through k . Then go 1 through $k + 1$. Then go 2 through $k + 2$... At each step we are adding at most $\max_i(w_i) - \min_i(w_i)$ to the sum so we will eventually enter $[L, U]$ at which point we stop.
- If no hits, output empty.
- We can trivially check that no sum we may reach lies outside the union of $[mi_k, ma_k]$ over all k

<https://oj.uz/submission/1095396>

Problem 3 (16IOI5)

Let $f(i)$ be where i goes after the permutation.

We claim that, using up to 128 elements in S and 128 queries, we may always narrow down what $f^{-1}(i)$ can be by $\frac{1}{2}$ for each i . We do this using induction,

THIS IS A SKETCH

Base case: We add 64 elements to S as follows,

$$\{[1 \text{ if } i = k \text{ else } 0 \text{ for } i \text{ in range}(128)] \text{ for } k \text{ in range}(64)\}$$

We send 128 queries $\{[1 \text{ if } i = k \text{ else } 0 \text{ for } i \text{ in range}(128)] \text{ for } k \text{ in range}(128)\}$. For the k -th query, if the answer we get is yes, we know $f^{-1}(k)$ is in the first 64. Otherwise, $f^{-1}(k) > 64$.

Note that, we know the image of $[1, 1, \dots, 1, 0, 0, \dots, 0]$ since we know which k received a 1.

Inductive step: We next send out queries of the form $[1, 1, 1, \dots, 1, \mathbf{0, 0, 1, 0, 1, 1, 1} \dots]$ where the length of the bolded section halves each time. We know the image of the non-bolded part so we send queries with those filled to one. We pull the same trick with asking queries on the uncertain bits. (This is a sketch so uh pls trust proof by AC. I also verbalized a similar idea better in Problem 11)

Note that the number of one increases every step so we won't have anything collide.

<https://oj.uz/submission/1095677>

8 clubs; 8 hours

Problem 5 (15HMMTT6)

We pick quarters, dimes, nickels, pennies in order where we move onto the smaller denomination from a type of coins if we run out of that type of coins or if adding another coin of that type will overflow our running sum. Once we are done with pennies, we stop.

If, at this point, we have \$50, we are done.

Consider the case where we are not.

Sketch: We may use mods to quickly prove we must have ran out of nickels and pennies at this point. We may then prove that we cannot also be out of dimes or out of quarters. Therefore, we must be exactly 5 cents short. As a result, by modular arithmetic, there must be at least three dimes left. Since we started with picking quarters and still have one left, we must have picked a quarter already. We remove one of the quarters we picked and take three more dimes to finish.

Problem 8 (20EGMO4)

For a given *fresh* permutation f of $\{1, 2, \dots, n-1\}$, we may insert an n before the 1, the 2, the 3, ..., or the $n-1$ to create $n-1$ permutations of $\{1, \dots, n\}$, which are each clearly *fresh*. In total, we have $(n-1)f_{n-1}$ *fresh-A* permutations of $\{1, \dots, n\}$ so far, all distinct.

For each permutation f , we may create one more *fresh* permutation by,

1. Replace $n-1$ with n .
2. Add $n-1$ at the end.

Since n is at the beginning, no prefix would be a permutation of $\{1, \dots, k\}$ as $k < n$ meaning this is indeed *fresh*. We call these the f_{n-1} *fresh-B* permutations.

We claim that each *fresh-B* is distinct with *fresh-A* as performing step 1 makes f no longer *fresh* since the first $n-2$ terms are now a permutation of $\{1, \dots, n-2\}$. Since all the *fresh-A* are still *fresh* after removing the n , they are distinct.

All *fresh-B* are distinct from each other as well. We may uniquely reverse engineer f by removing the $n - 1$ and then replacing the n with $n - 1$. Since the f were distinct to begin with, the images are also distinct.

In total, we have at least nf_{n-1} distinct *fresh* permutations over A and B .

13 clubs; 10.5 hours

Problem 11 (19HMIC2)

We perform the following algorithm.

Define a function $\text{calc}(l, r)$ that, knowing the subset S of $\{1, \dots, 2019\}$ that gets sent to $[l, r]$ under π , we may find out which terms of S will get sent to $\left[l, \left\lfloor \frac{l+r}{2} \right\rfloor\right]$ and $\left[\left\lfloor \frac{l+r}{2} \right\rfloor + 1, r\right]$ respectively in at most $|[l, r]| = r - l + 1$ moves for all $r - l \geq 2$.

Let $S[0], S[1], \dots, S[r - l]$ be the elements of S . We send out $r - l$ requests $f_1, f_2, \dots, f_{r-l}$ as follows,

$$f_i(x) = \begin{cases} S[0] & \text{for } x \in \left[l, \left\lfloor \frac{l+r}{2} \right\rfloor\right] \\ S[i] & \text{for } x \in \left[\left\lfloor \frac{l+r}{2} \right\rfloor + 1, r\right] \\ S[0] & \text{for all others} \end{cases}$$

We calculate how many indices match up.

Define $r_j = \pi^{-1}(S[j])$ for all j .

For $j = 0$, then $r_0 \in \left[l, \left\lfloor \frac{l+r}{2} \right\rfloor\right] \Leftrightarrow f_i(r_0) = S[0] \Leftrightarrow$ we have a match at r_0 .

For $j = i$, then $r_i \in \left[\left\lfloor \frac{l+r}{2} \right\rfloor + 1, r\right] \Leftrightarrow f_i(r_i) = S[i] \Leftrightarrow$ we have a match at r_i .

For all other j , since $\pi(r_j) = S[j]$ is not in the range of $f_i(x)$, we may never have a match at r_j . For all x such that $\pi(x) \notin S$, note that since $f_i(x) \in S$ for $x \notin [l, r]$, $f_i(x) \neq \pi(x)$ for such x . Therefore, the other terms will not add to number of indices that match up.

If the number of indices that match up is always greater than 1 meaning $f_i(x) \geq 1$ for all i , then we must always have a match at r_0 since we don't always have a match at r_i . This means $r_0 \in \left[l, \left\lfloor \frac{l+r}{2} \right\rfloor\right]$. Otherwise, $r_0 \in \left[\left\lfloor \frac{l+r}{2} \right\rfloor + 1, r\right]$. Using this, we then get whether $r_i \in \left[\left\lfloor \frac{l+r}{2} \right\rfloor + 1, r\right]$ for all i as desired.

We may recursively apply $\text{calc}(1, 2019)$ then $\text{calc}(1, 1010)$, $\text{calc}(1011, 2019)$ and so on, applying $\text{calc}\left(\left[l, \left\lfloor \frac{l+r}{2} \right\rfloor\right]\right)$ and $\text{calc}\left(\left[\left\lfloor \frac{l+r}{2} \right\rfloor + 1, r\right]\right)$ after $\text{calc}(l, r)$ if $r - l \geq 2$. If $r - l < 2$, then the calc function has given us which term in S gets sent to l, r respectively. **Note that the sum of the ranges is $2019 \cdot \lceil \log_2(2019) \rceil < 24000$, summing by depth. [clarification needed?]**

Problem 12 (10IMO5)

Let $(b_1, b_2, b_3, \dots, b_6)$ denote the number of coins in each box.

Lemma 1: $(n, 0, 0) \rightarrow (0, 2^n, 0)$ is possible for positive n .

Proof: For $n = 1, 2$, we do $(n, 0, 0) \rightarrow (n, 2n, 0)$ to finish. For greater n , we start with,

$$(n, 0, 0) \rightarrow (n-1, 2, 0) \rightarrow (n-1, 0, 4) \rightarrow (n-2, 4, 0)$$

Then we repeat $(n-k, 2^k, 0) \rightarrow (n-k, 0, 2^{k+1}) \rightarrow (n-k-1, 2^{k+1}, 0)$ to get the desired result. $\square$

With lemma 1 proven, note that we may get $(n, m, 0, 0) \rightarrow (n, 0, 2^m, 0) \rightarrow (n-1, 2^m, 0, 0)$. For our first steps, we do,

$$(1, 1, \dots, 1) \rightarrow (0, 3, 0, 3, 0, 3) \rightarrow (0, 3, 0, 2, 3, 0) \rightarrow (0, 2, 2, 0, 3, 0) \rightarrow (0, 2, 1, 3, 0, 0)$$

We now apply $(n, m, 0, 0) \rightarrow (n-1, 2^m, 0, 0)$,

$$\rightarrow (0, 2, 0, 8, 0, 0) \rightarrow (0, 1, 8, 0, 0, 0) \rightarrow (0, 0, 10, 0, 0, 0) \rightarrow (0, 0, 9, 2, 0, 0)$$

We now apply $(n, m, 0, 0) \rightarrow (n-1, 2^m, 0, 0)$ repeatedly,

$$(0, 0, 9, 2, 0, 0) \rightarrow (0, 0, 8, 4, 0, 0) \rightarrow (0, 0, 7, 16, 0, 0) \rightarrow \dots$$

$$\rightarrow \left(0, 0, 0, 2^{2^{2^{2^{2^{2^{2^2}}}}}}, 0, 0 \right)$$

Note that, clearly, $2^{2^{2^{2^{2^{2^{2^2}}}}} = 2^{2^{2^{2^{2^{2^{65536}}}}} > 2^{2^{2^{2^{2^{2010^{2010}}}}} > \frac{2010^{2010^{2010}}}{4}$. Therefore, we switch the last two zeroes until,

$$\rightarrow \left(0, 0, 0, \frac{2010^{2010^{2010}}}{4}, 0, 0 \right) \rightarrow (0, 0, 0, 0, 0, 2010^{2010^{2010}})$$

21 clubs; 13 hours

Problem 14 (03AMO6)

Let (a, b, c, d, e, f) denote a certain configuration by the value of each vertex. We consider rotations and reflections equivalent to each other.

Call the configuration $(0, x, x, 0, x, x)$ for positive x “dead” and all other configurations “alive”.

Lemma: Each number in the configuration is always non-negative.

Proof: At no step will we get a negative number. $\square$

We claim that, at any alive configuration with positive sum, we may use a finite number of moves to get to a configuration with a strictly smaller sum which is also alive.

Case 1: The configuration is of the form $(0, a, x, 0, x, x)$ for $a \neq x$.

If $x = 0$, $(0, a, 0, 0, 0, 0) \rightarrow (0, 0, 0, 0, 0, 0)$.

If $a < x$, we do $(0, a, x, 0, x, x) \rightarrow (0, a, a, 0, x, x)$ to suffice.

If $a > x$, we alternate applying the operation on the first and second positions,

$$(0, a, x, 0, x, x) \rightarrow (a - x, a, x, 0, x, x) \rightarrow (a - x, |2x - a|, x, 0, x, x)$$

FTSOC, if, following this process, the sum never gets smaller, we induct to see $a - kx < 0$ for all positive k .

Base case: If $2x - a \geq 0$, then the sum $(a - x) + (2x - a) + x + 0 + x + x = 4x$ got smaller.

Inductive step: Since $(k - 1)x - a \geq 0$, depending on the parity of k , we do one of,

$$(a - (k - 1)x, a - (k - 2)x, x, 0, x, x) \rightarrow (a - (k - 1)x, |a - kx|, x, 0, x, x)$$

$$(a - (k - 2)x, a - (k - 1)x, x, 0, x, x) \rightarrow (|a - kx|, a - (k - 1)x, x, 0, x, x)$$

If $kx - a \geq 0$, then $|a - kx| = kx - a$ and the sum of the terms would be $4x$ which got smaller (!). Therefore, $kx - a < 0$.

Since a is a finite positive number, the only way is for $x = 0$ which we already covered. [End of case]

Case 2: The configuration is of the form $(0, a, a, 0, x, x)$ for $a \neq x$.

WLOG, let $a < x$.

Subcase 1: $a = 0$

$$(0, 0, 0, 0, x, x) \rightarrow (0, x, x, x, x, x) \rightarrow (0, x, 0, x, 0, x) \rightarrow (0, 0, 0, 0, 0, 0)$$

Subcase 2: $a \neq 0$

$$(0, a, a, 0, x, x) \rightarrow (x - a, a, a, x - a, x, x) \rightarrow (x - a, a, a, x - a, a, a) \rightarrow (0, a, a, x - a, a, a)$$

The sum went from $2x + 2a$ to $x + 3a$ and got smaller as desired. [End of case]

Case 3: All other configurations.

Let m be the maximum number in the configuration. Note that if the positive difference of m 's neighbors is less than m , we are done in one move. Assume that this is not the case.

By the lemma and using m maximum, one must be zero and one must be another m .

Looking at m 's neighbor which has value m as well, applying the same logic we get m 's neighbor's other neighbor must be zero if we aren't done in one move. In this case, we have the configuration is $(0, m, m, 0, x, y)$.

If $x \neq y$, then WLOG $x > y$ meaning $(0, m, m, 0, x, y) \rightarrow (0, m, m, 0, y, y)$ is smaller. If $x = y$, then we have case 2 which is already solved. [End of case]

Since 2003^{2003} is odd, we could not have started "dead". Therefore, our sum will eventually go to zero as desired.

Problem 16 (22IMO6)

We claim the answer is $1 + 2n(n - 1)$.

We first prove the bound. Note that the number 1 must be at a valley on the grid, meaning we have one path of 1 vertex starting and ending there. There are $2n(n - 1)$ distinct pairs of adjacent cells in the grid ($n(n - 1)$ pairs adjacent horizontally and an equal amount adjacent vertically). For each of these pairs, we may find a path which end with these two cells by reverse-engineering the path, going to a lower adjacent square every move until we hit a valley. Since the finish of these paths are distinct and each path has length greater than 1, we have $1 + 2n(n - 1)$ paths constructed as desired.

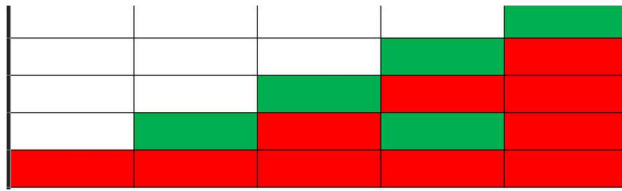
We now show attainability. Note that this is possible if we find a construction of a square where there is only one "valley" and, for every pair of adjacent cells, there is exactly one path that ends with those two. In this case, there would only be one path of length 1 and all other paths must end at distinct pairs of cells of which there are $2n(n - 1)$ options.

Lemma: We may color a $n \times n$ square red and green such that,

- No two green cells are adjacent.
- For every pair of red cells, there is **exactly** one path connecting them which passes through other adjacent red cells (i.e. the red cells lie on a connected tree)

Proof: Denote the cells to be (x, y) for $1 \leq x, y \leq n$. For our construction, we color $(1, 1)$ red and (x, x) green for all $2 \leq x \leq n$. This way, we may focus on coloring cells of the form (x, y) and $x > y$ then reflect our result over the $y = x$ line to finish.

As an example, for $n = 5$, if we figured out the coloring for the cells of form $x > y$,



We flip this over finish,



This is because for a cell to cross $y = x$, it must do so passing $(1, 1)$. We now use induction to finish the proof.

Base case: $n = 1, 2, \dots, 6$

The case $n = 5$ has been shown above. For $n \leq 4$, the left-bottommost sub-square of the following clearly suffices.



For $n = 6$, this clearly works,



Inductive step: $n \rightarrow n + 6$

We do the following:



We have two red diagonals that run to the right-most edge separated by green. Fill in the construction for n into the yellow to finish.

Conclusion: Using mathematical induction, we are done. $\square$

With the claim proven, we fill out the square. Choose any red cell as 1. We use breadth-first search outwards from that cell, numbering each new cell we discover with the next available number. For example, in the $n = 5$ case, if we set $(1, 1)$ to be the valley one,

10	12	14	17	
6		16		19
4	7		18	15
2		8		13
1	3	5	9	11

We now fill in the green cells with the rest of the numbers in arbitrary order.

10	12	14	17	22
6	20	16	21	19
4	7	23	18	15
2	25	8	24	13
1	3	5	9	11

We see that each of the green cell's neighbors are lower than itself since all green have greater value than the reds. Additionally, only one of each red cell's neighbors (apart from 1) are lower than itself, namely the neighbor which is in the direction of the 1 since we used BFS on a tree and cells further away from 1 have a higher value always.

There is a unique path ending at each pair of adjacent cells and we are done since, reverse engineering from the last edge, there is only one lower option to choose from at each red cell.

35 clubs; 16 hours

Problem 17 (19SLC8)

The cases for $n \leq 9$ are trivial since $4n$ questions are enough to ask about every vertex.

Assume $n \geq 10$. Rename town $\rightarrow$ vertex and road $\rightarrow$ edge.

We reveal the directions of the roads one by one. At any point, we color a vertex blue if no roads that are revealed lead away from it, yellow if one road leads away from it, and red if at least two roads lead away from it.

Stage 1: Color $n - 1$ vertices yellow.

1. Start at any vertex to set as the “position” vertex (the “position” is always blue)
2. Choose any other blue vertex.
3. Repeat the following steps until no blue vertices other than the position is left.
 - a. Reveal the edge between the “position” to the other blue vertex (this edge cannot already be revealed since both vertices are blue). Either the “position” or the other vertex becomes yellow.
 - b. Have the “position” move to or stay at the vertex that stays blue.

Step 3 will end after $n - 1$ reveals at which time $n - 1$ vertices are yellow and only 1 blue vertex is left. We show by induction that the graph composing of the revealed edges forms a tree after each iteration of step 3: each reveal must introduce a fresh new vertex disconnected from the existing tree by connecting it to the “position” vertex on the tree.

Using the fact that it’s a tree, we go into stage 2 with the knowledge that the edge between two vertices is only already revealed if they were adjacent on the tree.

Stage 2: Color $n - 2$ vertices red.

Let $v_1, v_2, \dots, v_m$ be the vertices adjacent to “position” on the tree.

Repeat the following until either all vertices except $v_1, \dots, v_m$ and “position” are red, or “position” is red: We choose a yellow vertex outside of $v_1, \dots, v_m$ and “position” to reveal

the edge from it to “position”. The color of “position” will advance from blue to yellow or yellow to red, or the color of the other vertex will go to red.

Case 1: “Position” is still blue or yellow after all of this.

We are done. Every edge adjacent to “position” has been revealed and “position” still isn’t red meaning there is a vertex with outdegree at most 1. Stage 1 used $n - 1$ moves and stage 2 used at most $n - 1$ moves so far since we are just revealing edges adjacent to “position”, so we are comfortably within the $4n$ question limit.

Case 2: “Position” is red.

Every vertex on the graph is now yellow or red. We remove all red vertices from the tree we drew as well as the edges adjacent to this. We do this after coloring any new vertex red. We also continue marking revealed edges. After the removal, the graph is a forest.

We do the following step until stuck: Find a pair of two yellow vertices that are not adjacent to each other. We reveal the edge between them to color one of them red.

Note that, after coloring one of the vertices red and removing it from the graph, we have removed the newly added edge as well. Therefore, the graph remains a forest. Since the graph is a forest, we will be able to do the step until there are one or two yellow vertices left.

Stage 3 (only for case 2):

Note that every step we take from stage 1 or 2, we either change a blue vertex to yellow or a yellow vertex to red.

Case a: There are two yellow vertices left.

We have taken $2n - 2$ steps so far. We reveal all the edges adjacent to the two yellow vertices to finish.

Case b: There is one yellow vertex left.

We have taken $2n - 1$ steps so far. We reveal all the edges adjacent to the last yellow vertex to finish.

Problem 21 (CF763B)

For each rectangle, move its (x_1, y_1) to its top-left corner and (x_2, y_2) to its bottom-right.

If x_1 is odd and y_1 is odd, color red.

If x_1 is even and y_1 is odd, color blue.

If x_1 is odd and y_1 is even, color green.

If x_1 is even and y_1 is even, color purple.

For two rectangles to be touching, since all sides are odd, they cannot have the same color.

```
N = int(input())
print("YES")
for i in range(N):
    a, b, c, d = map(int, input().split(" "))
    print(2 * (max(a, c) % 2) + (max(b, d) % 2) + 1)
```

This problem took me embarrassingly long; I tried for so long to BFS outwards somehow.

52 clubs; 20 hours

Problem 23 (CF1270G)

We set $a'_i = i - a_i$.

We search for a subset $\{a_{k_1}, a_{k_2}, \dots, a_{k_m}\}$ such that its sum is zero. Equivalently, the corresponding subset $\{a'_{k_1}, a'_{k_2}, \dots, a'_{k_m}\}$ satisfies,

$$\sum_{i=1}^m k_i = \sum_{i=1}^m a'_{k_i}$$

To find this subset, we construct a graph where the vertices are $1, \dots, n$ and we draw a directed edge from i to a'_i . This is possible since $i - n \leq a_i \leq i - 1 \Leftrightarrow n \geq a'_i \geq 1$. If we follow the graph starting at vertex 1, we must eventually revisit a vertex we've already visited since there are only n vertices in total. This way, we'd find a directed loop on the graph. We use the numbers on this loop to finish.

Problem 24 (CF1325F)

We may find a maximum independent set as follows:

1. Locate the vertex with the minimum degree on the graph
2. Delete all the neighbors of that vertex
3. Add the vertex to the independent set then delete it
4. Repeat the process until graph is empty or if we deleted a vertex on the graph with degree at least $\sqrt{n} - 1$.

If we follow the process to the end, we deleted at most $\sqrt{n}$ vertices each move, meaning we have at least $\sqrt{n}$ vertices in our set.

If we deleted a vertex with degree $\sqrt{n} - 1$, each vertex on the graph must have had degree $\sqrt{n} - 1$ prior to the deletion. We call this subgraph of the original graph G . We follow the algorithm below to find a cycle of length at least $\sqrt{n}$.

We now perform a DFS on G in an arbitrary direction until we are forced to take a step back when we encounter a vertex v_{end} where all its neighbors are visited. At this point, since v_{end} has $\sqrt{n} - 1$ neighbors, we have found a path with at least $\sqrt{n}$ vertices. We travel backwards on the path until we reach the one v of the $\sqrt{n} - 1$ neighbors of v_{end} that is at a maximal distance from v_{end} following the path. Since v and v_{end} are adjacent via both an edge and a path of $\sqrt{n} - 1$ length (noting that $n \geq 5$), we found a cycle with length at least $\sqrt{n}$.

<https://codeforces.com/contest/1325/submission/286101452>

62 clubs; 24 hours

Mini Survey

I feel like Problem 16 (22IMO6) should be a 9-clubber because of its difficulty especially since it is more difficult, interesting, and educational than 03AMO6 in my opinion.